TP 2.1 - GENERADORES PSEUDOALEATORIOS

Brizio Augusto
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
brizioaugusto@gmail.com

Casermeiro Gonzalo Nicolás
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
gonzacasermeiro@gmail.com

Messina Tiziano Leonel
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
tizilmessina@gmail.com

De la Rosa Valentin
Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Rosario, Argentina
mferreyra079@gmail.com

June 4, 2026

ABSTRACT

Este trabajo estudia la generación de números pseudoaleatorios mediante la implementación y comparación de tres fuentes de datos: un generador congruencial lineal (GCL), el método de los cuadrados medios y el generador provisto por Python mediante `random.random()`. Para evaluar la calidad estadística de las secuencias generadas se aplicaron seis pruebas: test de media, test de varianza, prueba chi-cuadrado de uniformidad, prueba de Kolmogorov-Smirnov, prueba de corridas y autocorrelación de primer orden.

Se generaron $n = 10\,000$ valores para cada generador utilizando una semilla fija para garantizar reproducibilidad. Los resultados muestran que el GCL implementado y el generador de Python superan todas las pruebas consideradas, mientras que el método de cuadrados medios presenta fallas significativas en uniformidad, media y varianza. El análisis confirma que no alcanza con producir valores aparentemente desordenados: un generador útil para simulación debe aproximar una distribución uniforme, evitar patrones detectables y sostener esas propiedades a lo largo de secuencias suficientemente largas.

1 Introducción

La simulación computacional requiere representar procesos inciertos mediante secuencias numéricas que se comporten como si fueran aleatorias. En una computadora, sin embargo, la aleatoriedad pura no surge naturalmente: los valores son generados por algoritmos deterministas que, a partir de un estado inicial o *semilla*, producen una secuencia reproducible. A estos algoritmos se los conoce como generadores de números pseudoaleatorios.

La reproducibilidad es una ventaja central en simulación, ya que permite repetir un experimento bajo las mismas condiciones iniciales y verificar los resultados. Pero también introduce un riesgo: si el algoritmo posee ciclos cortos, sesgos o correlaciones internas, la simulación puede producir conclusiones equivocadas. Por este motivo, además de programar generadores, es necesario evaluarlos con pruebas estadísticas que permitan detectar desviaciones respecto del comportamiento esperado.

El objetivo de este trabajo es implementar generadores pseudoaleatorios en Python 3.x, aplicar tests de calidad sobre las secuencias obtenidas y comparar su desempeño con el generador incluido en el lenguaje. El informe se organiza de la siguiente manera: la Sección 2 describe los generadores estudiados; la Sección 3 presenta las pruebas estadísticas utilizadas; la Sección 4 expone los resultados obtenidos; y la Sección 5 desarrolla las conclusiones.

2 Generadores estudiados

2.1 Generadores aleatorios reales

Los generadores aleatorios reales obtienen valores a partir de fenómenos físicos no deterministas o muy difíciles de predecir, como ruido atmosférico, ruido térmico, decaimiento radiactivo o dispositivos de hardware especializados. A diferencia de los generadores pseudoaleatorios, no dependen únicamente de una semilla ni producen necesariamente la misma secuencia al repetir el experimento.

Su principal ventaja es que pueden aportar entropía real. Sin embargo, para simulaciones computacionales presentan algunas limitaciones prácticas: suelen ser menos reproducibles, pueden requerir dispositivos externos o conexión a servicios de terceros, y no siempre permiten repetir exactamente una corrida bajo las mismas condiciones iniciales. Por este motivo, en simulación suele ser preferible trabajar con generadores pseudoaleatorios de buena calidad, ya que permiten controlar la semilla y repetir los experimentos. En este trabajo, los generadores aleatorios reales se consideran como referencia conceptual, mientras que la implementación se concentra en generadores pseudoaleatorios reproducibles.

2.2 Generador congruencial lineal

El generador congruencial lineal (GCL) es uno de los métodos clásicos para obtener números pseudoaleatorios. A partir de una semilla inicial X_0, la secuencia se define mediante la recurrencia:

$$X_{i+1} = (aX_i + c) \bmod m \tag{1}$$

donde a es el multiplicador, c el incremento y m el módulo. Para trabajar con valores en el intervalo $[0, 1)$, cada entero generado se normaliza:

$$U_i = \frac{X_i}{m} \tag{2}$$

En la simulación se utilizaron los parámetros:

$$a = 1103515245, \qquad c = 12345, \qquad m = 2^{31} \tag{3}$$

con semilla $X_0 = 5731$. Estos parámetros corresponden a una familia de GCLs ampliamente utilizada históricamente. Si bien los GCLs modernos no son recomendables para criptografía, siguen siendo útiles para introducir los principios de generación pseudoaleatoria y para comparar calidad estadística.

2.3 Método de los cuadrados medios

El método de los cuadrados medios fue propuesto por John von Neumann. La idea es elevar al cuadrado la semilla actual, completar con ceros si es necesario y tomar los dígitos centrales como nuevo estado:

$$X_{i+1} = \text{dígitos centrales}(X_i^2) \tag{4}$$

Luego se normaliza el valor dividiéndolo por 10^d, donde d es la cantidad de dígitos utilizada:

$$U_i = \frac{X_i}{10^d} \tag{5}$$

En este trabajo se emplearon semillas de cuatro dígitos con $d = 4$ y semilla inicial 5731. Aunque el método es simple, suele presentar ciclos cortos, convergencia a cero o patrones repetitivos, por lo que se espera un rendimiento inferior al del GCL y al generador de Python.

2.4 Generador de Python

Como referencia se utilizó `random.random()` de Python, que genera valores en $[0, 1)$ mediante el generador Mersenne Twister. Este generador tiene un período muy grande y buenas propiedades estadísticas para simulación general, aunque no debe utilizarse para fines criptográficos [5].

3 Pruebas estadísticas

Para cada generador se produjeron $n = 10\,000$ valores pseudoaleatorios. Las secuencias fueron sometidas a seis pruebas. En todas ellas se tomó un nivel de significancia aproximado de $\alpha = 0{,}05$, equivalente al criterio usual $|Z| < 1{,}96$ para pruebas bilaterales.

3.1 Test de media

Para una distribución uniforme $U(0, 1)$, la esperanza teórica es:

$$E[U] = \frac{1}{2} \tag{6}$$

El estadístico utilizado fue:

$$Z_{\bar{U}} = \frac{|\bar{U} - 0{,}5|}{\sqrt{\frac{1}{12n}}} \tag{7}$$

y se acepta el comportamiento esperado si $Z_{\bar{U}} < 1{,}96$.

3.2 Test de varianza

La varianza teórica de una distribución uniforme $U(0, 1)$ es:

$$Var(U) = \frac{1}{12} \tag{8}$$

Se comparó la varianza muestral contra este valor esperado mediante una aproximación normal del estadístico chi-cuadrado:

$$Z_{S^2} = \frac{\left| \frac{(n-1)S^2}{1/12} - (n-1) \right|}{\sqrt{2(n-1)}} \tag{9}$$

3.3 Prueba chi-cuadrado

El intervalo $[0, 1)$ se dividió en $k = 10$ subintervalos de igual amplitud. Para cada intervalo se comparó la frecuencia observada O_j contra la frecuencia esperada $E_j = n/k$:

$$\chi^2 = \sum_{j=1}^{k} \frac{(O_j - E_j)^2}{E_j} \tag{10}$$

Con $k - 1 = 9$ grados de libertad, el valor crítico para $\alpha = 0{,}05$ es $16{,}919$. La secuencia supera la prueba si $\chi^2 < 16{,}919$.

3.4 Prueba de Kolmogorov-Smirnov

La prueba de Kolmogorov-Smirnov compara la distribución empírica $F_n(x)$ con la función de distribución acumulada teórica $F(x) = x$:

$$D = \sup_x |F_n(x) - F(x)| \tag{11}$$

Para $n = 10\,000$, se utilizó el criterio:

$$D < \frac{1{,}36}{\sqrt{n}} = 0{,}01360 \tag{12}$$

3.5 Prueba de corridas

La prueba de corridas analiza la alternancia de valores por encima y por debajo de $0{,}5$. Si los valores son independientes, la cantidad de corridas debe aproximarse a:

$$E[R] = \frac{2n_1n_2}{n_1 + n_2} + 1 \tag{13}$$

donde n_1 es la cantidad de valores superiores o iguales a $0{,}5$ y n_2 la cantidad de valores inferiores. La varianza de la cantidad de corridas se aproxima mediante:

$$Var(R) = \frac{2n_1n_2(2n_1n_2 - n_1 - n_2)}{(n_1 + n_2)^2(n_1 + n_2 - 1)} \tag{14}$$

Con esto, el estadístico utilizado fue:

$$Z_R = \frac{|R - E[R]|}{\sqrt{Var(R)}} \tag{15}$$

y se acepta la ausencia de patrones significativos en las corridas si $Z_R < 1{,}96$.

3.6 Autocorrelación

Para detectar dependencia lineal entre valores consecutivos se calculó la autocorrelación de primer orden:

$$\rho_1 = \frac{\sum_{i=1}^{n-1}(U_i - \bar{U})(U_{i+1} - \bar{U})}{\sum_{i=1}^{n}(U_i - \bar{U})^2} \tag{16}$$

Luego se evaluó el estadístico $Z = |\rho_1|\sqrt{n}$, aceptando la hipótesis de ausencia de autocorrelación lineal si $Z < 1{,}96$.

4 Resultados y discusión

La Tabla 1 resume los resultados obtenidos para los tres generadores. El valor **OK** indica que el estadístico quedó dentro del rango esperado; **RECHAZA** indica evidencia suficiente para descartar el comportamiento esperado en esa prueba.

Table 1: Resultados de las pruebas estadísticas para $n = 10\,000$ valores.

Generador	**Test**	**Estadístico**	**Criterio**	**Resultado**
GCL	Media	0,04387	$< 1{,}96$	OK
GCL	Varianza	0,01743	$< 1{,}96$	OK
GCL	Chi-cuadrado	3,14800	$< 16{,}919$	OK
GCL	Kolmogorov-Smirnov	0,00492	$< 0{,}01360$	OK
GCL	Corridas	0,99737	$< 1{,}96$	OK
GCL	Autocorrelación	1,25931	$< 1{,}96$	OK
Cuadrados medios	Media	3,32149	$< 1{,}96$	RECHAZA
Cuadrados medios	Varianza	27,99503	$< 1{,}96$	RECHAZA
Cuadrados medios	Chi-cuadrado	14795,75200	$< 16{,}919$	RECHAZA
Cuadrados medios	Kolmogorov-Smirnov	0,20780	$< 0{,}01360$	RECHAZA
Cuadrados medios	Corridas	0,05990	$< 1{,}96$	OK
Cuadrados medios	Autocorrelación	0,18212	$< 1{,}96$	OK
Python random	Media	0,88573	$< 1{,}96$	OK
Python random	Varianza	0,23982	$< 1{,}96$	OK
Python random	Chi-cuadrado	4,69600	$< 16{,}919$	OK
Python random	Kolmogorov-Smirnov	0,00903	$< 0{,}01360$	OK
Python random	Corridas	0,43448	$< 1{,}96$	OK
Python random	Autocorrelación	0,12095	$< 1{,}96$	OK

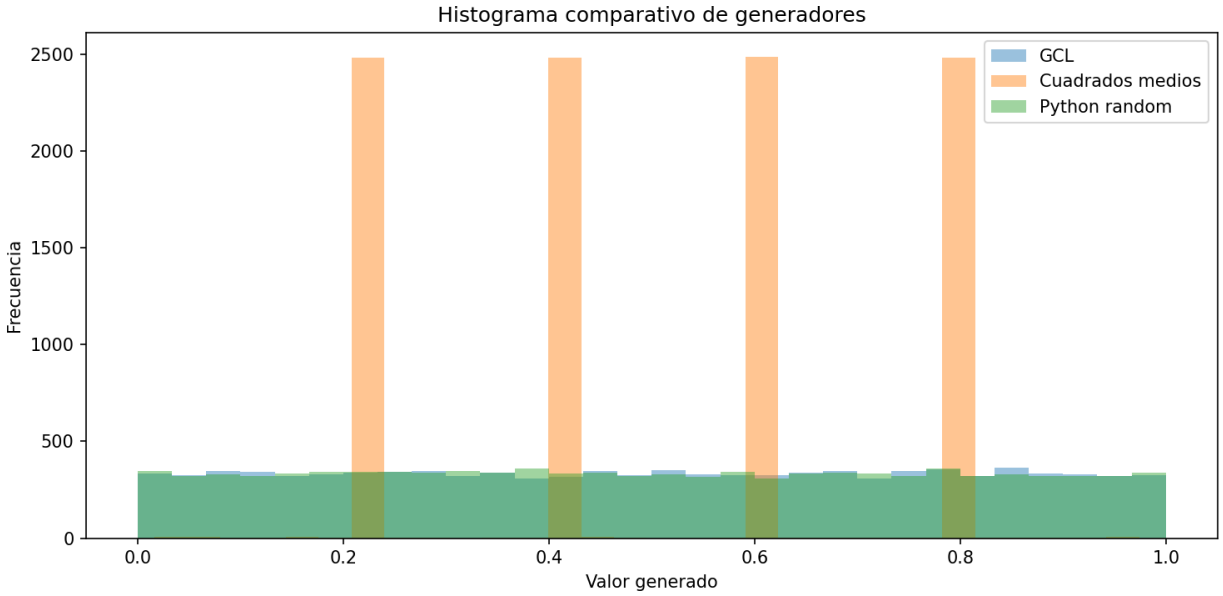


Figure 1: Histograma comparativo de los valores generados por cada método.

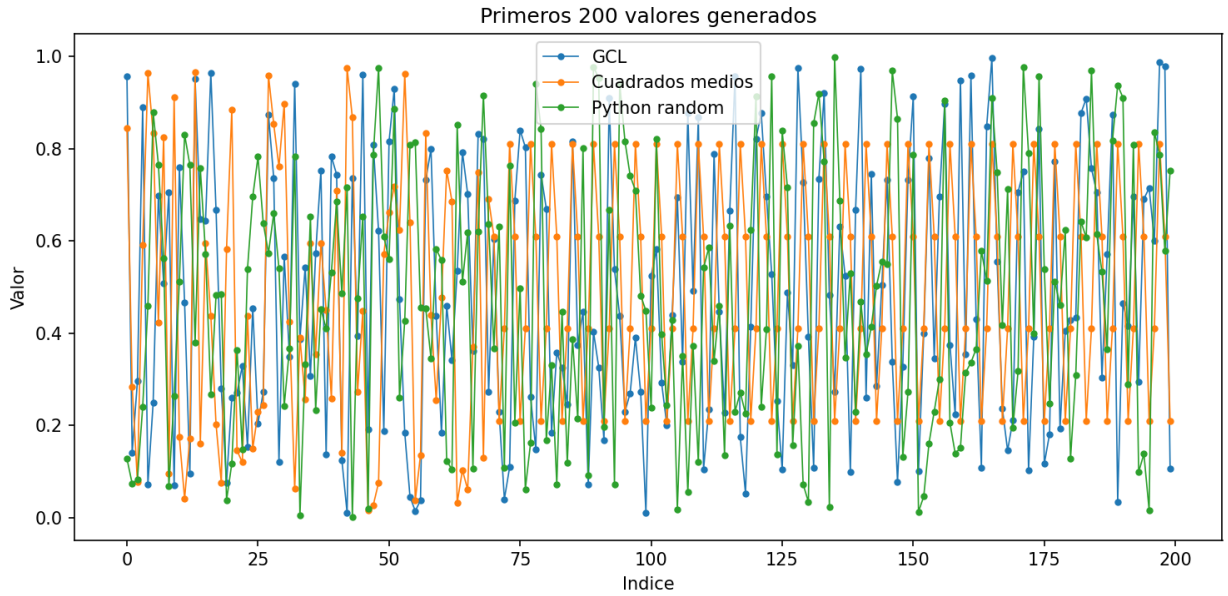


Figure 2: Primeros 200 valores generados por cada método.

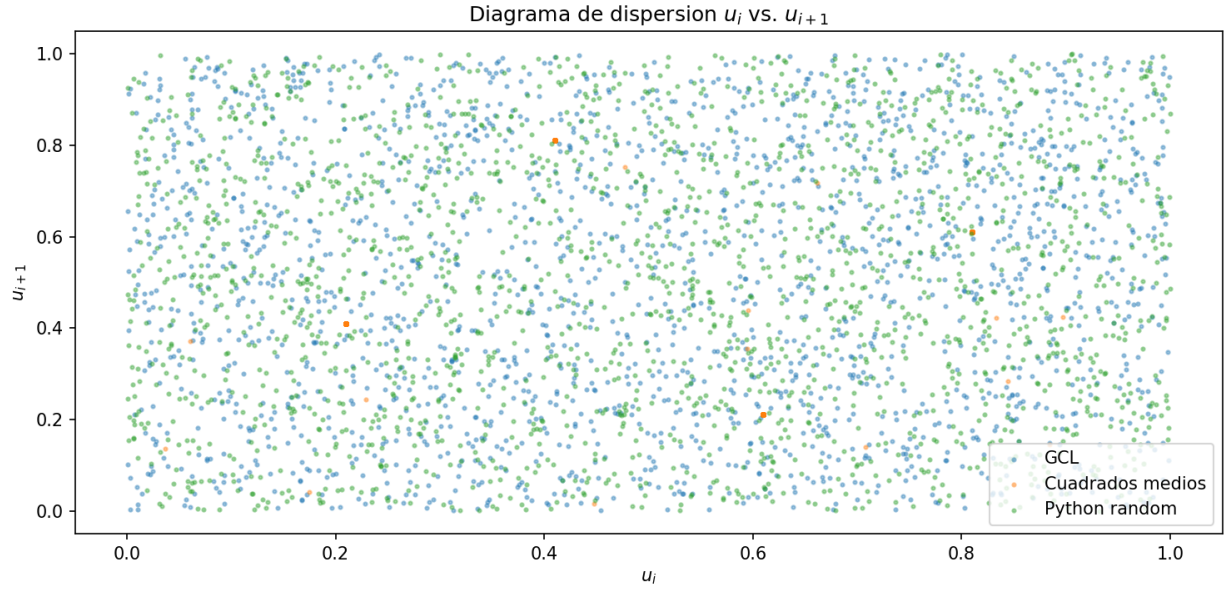


Figure 3: Diagrama de dispersión entre valores consecutivos U_i y U_{i+1}.

4.1 Análisis del GCL

El generador congruencial lineal superó las seis pruebas aplicadas. La media y la varianza se mantuvieron muy cerca de los valores teóricos de una uniforme $U(0,1)$, mientras que las pruebas de uniformidad global también quedaron dentro de los límites esperados. El valor de chi-cuadrado fue $3{,}14800$, muy por debajo del límite $16{,}919$, lo que indica una distribución equilibrada entre los subintervalos.

La prueba de corridas y la autocorrelación tampoco detectaron patrones fuertes entre valores consecutivos. Esto no significa que el GCL sea perfecto: los GCLs pueden presentar estructuras en dimensiones superiores y no son aptos para criptografía. Sin embargo, para el tamaño de muestra y las pruebas usadas en este trabajo, su comportamiento fue aceptable para simulación básica.

4.2 Análisis del método de cuadrados medios

El método de cuadrados medios mostró el peor desempeño. Falló el test de media, el test de varianza, la prueba chi-cuadrado y la prueba de Kolmogorov-Smirnov. El rechazo por chi-cuadrado fue especialmente marcado, con un $14795{,}75200$, muy superior al valor crítico $16{,}919$. Esto indica que los valores no se distribuyeron uniformemente en el intervalo $[0,1)$.

Aunque superó la prueba de corridas y la prueba de autocorrelación, esto no alcanza para considerarlo un generador adecuado. Una secuencia puede alternar razonablemente alrededor de $0{,}5$ y aun así estar concentrada en ciertas zonas del intervalo o tener una varianza incorrecta. El resultado confirma la debilidad histórica del método: es simple de implementar, pero muy sensible a la semilla y propenso a ciclos cortos o estados degenerados.

4.3 Análisis del generador de Python

El generador de Python superó todas las pruebas, como era esperable para una herramienta estándar de simulación general. Sus estadísticos se mantuvieron lejos de los valores críticos tanto en uniformidad como en independencia básica. En particular, la prueba chi-cuadrado obtuvo $4{,}69600$ y Kolmogorov-Smirnov $0{,}00903$, valores compatibles con una muestra uniforme.

Este comportamiento lo convierte en una alternativa confiable para simulaciones académicas y experimentales. No obstante, como indica la documentación oficial, el módulo `random` no debe utilizarse para aplicaciones criptográficas, ya que su objetivo es la simulación reproducible y no la seguridad.

5 Conclusiones

El trabajo permitió comprobar experimentalmente que la calidad de un generador pseudoaleatorio no puede asumirse únicamente porque sus valores parezcan desordenados. Es necesario aplicar pruebas estadísticas que evalúen uniformidad, media, varianza, independencia y ausencia de patrones simples.

El generador congruencial lineal implementado tuvo un desempeño satisfactorio en las seis pruebas realizadas. Esto muestra que, con una selección adecuada de parámetros, un GCL puede ser útil en simulaciones introductorias. Sin embargo, también se debe reconocer su limitación: existen pruebas más exigentes y análisis multidimensionales capaces de revelar patrones que los tests básicos no detectan.

El método de los cuadrados medios resultó claramente deficiente. Su falla en media, varianza y uniformidad confirma que se trata de un algoritmo histórico y didáctico, pero poco recomendable para simulaciones serias. La comparación con Python refuerza esta conclusión: el generador estándar del lenguaje mantuvo un comportamiento estable y superó todas las pruebas propuestas.

En síntesis, para simulaciones reales conviene utilizar generadores probados y bien documentados. Los métodos simples como cuadrados medios son valiosos para comprender el problema, pero no para reemplazar generadores modernos. El GCL, por su parte, funciona correctamente en este experimento y permite observar de manera clara la relación entre algoritmo, semilla, período y calidad estadística.

References

[1] J. I. Torres, *TP 2.1 - Generadores pseudoaleatorios*, Universidad Tecnológica Nacional - FRRO, 2026.

[2] Tereom, *Números pseudoaleatorios*, `https://tereom.github.io/est-computacional-2018/numeros-pseudoaleatorios.html`.

[3] Random.org, *Introduction to Randomness and Random Numbers*, `https://www.random.org/analysis/`.

[4] Random.org, *Analysis of the Random Number Generator*, `https://www.random.org/analysis/Analysis2005.pdf`.

[5] Python Software Foundation, *random — Generate pseudo-random numbers*, `https://docs.python.org/3/library/random.html`.

[6] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, Addison-Wesley, 1997.